

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

О Т Ч Е Т П О К У Р С О В О Й Р А Б О Т Е

на тему «Создание web-приложения Movie Picker»
по дисциплине «Конструирование программного обеспечения»

Выполнили студенты гр.
5130904/20101

Вдовина С. А.
Коба А. Ю.
Месропян А. В.

Руководитель

А.С. Иванов

«16» июня 2025 г.

Оглавление

Постановка задачи.....	4
Задачи, которые решает система.....	4
Аудитория:.....	4
Ключевые требования:.....	4
Архитектура приложения.....	5
Общая схема архитектуры.....	5
Frontend (React + TypeScript).....	5
Основные страницы:.....	5
Ключевые технологии:.....	6
Backend (Go + chi).....	6
Основные модули.....	6
Бизнес-логика разделена на уровни.....	6
База данных.....	6
Документация API (Swagger / OpenAPI).....	6
Взаимодействие с внешними API.....	7
Безопасность.....	7
Docker-инфраструктура.....	7
Ход работы.....	8
1. Постановка задачи.....	8
2. Проектирование архитектуры.....	8
3. Разработка backend-части.....	8
4. Разработка frontend-части.....	9
5. Интеграция с внешними сервисами.....	10
6. Тестирование и отладка.....	10
Unit - тесты.....	10
Интеграционные тесты.....	10
7. Контейнеризация и развертывание.....	10
8. Финальная доработка.....	11
Планы на будущее / Возможности расширения.....	13
1. Рекомендательная система.....	13
2. Фильтрация и сортировка.....	13
3. Комментарии и обсуждения.....	13
4. Избранное и история просмотров.....	13
5. Социальные функции.....	13
6. Адаптивная и мобильная версия.....	14
7. Административная панель.....	14
8. Интеграция сторонних API.....	14
9. Мультиязычность.....	14
10. Монетизация.....	14
Вывод.....	15
Соответствие поставленной задаче.....	15

Технические итоги.....	15
Полученные навыки и опыт.....	15
Заключение.....	15
Приложения.....	16
Скриншоты приложения.....	16

Постановка задачи

Целью разработки веб-приложения **Movie Picker** является создание интуитивно понятной платформы, позволяющей пользователям управлять личным каталогом фильмов: сохранять интересные фильмы, выставлять собственные оценки и получать доступ к обзорам с YouTube. Приложение ориентировано на пользователей, которые часто ищут, запоминают и оценивают фильмы, но не имеют единого инструмента для этого.

Задачи, которые решает система

1. **Хранение личного списка фильмов** ("Хочу посмотреть"):
 - Пользователь может добавлять фильмы в список к просмотру и удалять их оттуда.
 - Все данные хранятся на сервере, доступны после авторизации.
2. **Выставление пользовательских рейтингов**:
 - После просмотра фильма пользователь может оценить его по пятибалльной шкале.
 - Все оценки сохраняются и отображаются в отдельном разделе.
3. **Поиск фильмов**:
 - Реализован удобный поиск по названию.
 - Подгружаются популярные фильмы с внешнего источника при отсутствии запроса.
4. **Получение видеообзоров с YouTube**:
 - Для каждого фильма автоматически подбираются видеорецензии с YouTube.
 - Пользователь может перейти к ним прямо из приложения.
5. **Управление пользовательским профилем**:
 - Пользователь может изменить email или пароль на отдельной странице.
 - Для безопасности требуется ввод текущего пароля при изменениях.
6. **Регистрация и вход с авторизацией**:
 - Используется email и пароль.
 - Авторизация через JWT.

Аудитория:

- Люди, которые активно смотрят фильмы и хотят вести учёт.
- Те, кто сохраняет интересные фильмы "на потом" (вместо заметок или скриншотов).
- Любители сравнивать собственные оценки с впечатлениями из видеообзоров.

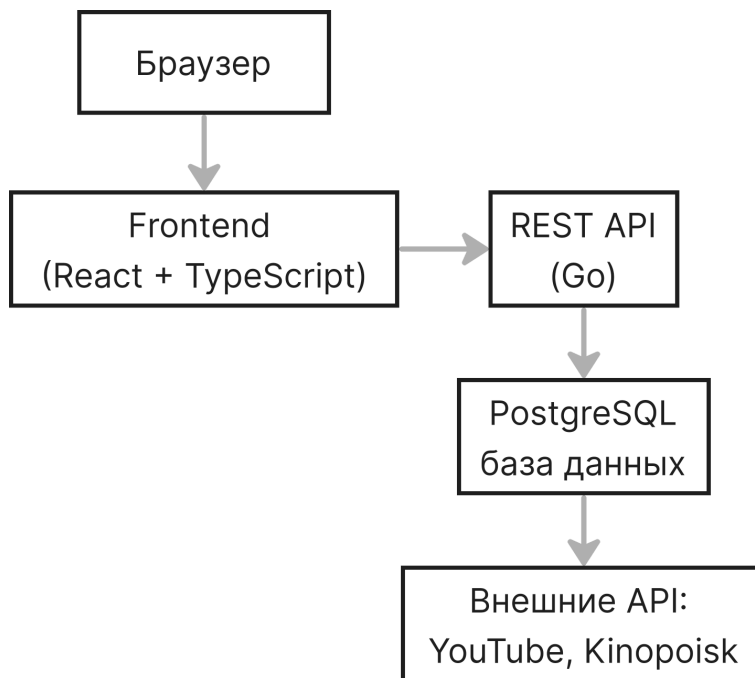
Ключевые требования:

- Удобный и быстрый интерфейс (SPA на React).
- Backend с защищённой авторизацией и API (на Go).
- Использование внешних источников данных (Kinopoisk, YouTube).
- Хранение пользовательских данных в базе (PostgreSQL).
- Масштабируемая архитектура (Docker-контейнеризация, чистая структура).

Архитектура приложения

Приложение **Movie Picker** построено по модели клиент–сервер и включает в себя два основных компонента: Frontend (клиентская часть) и Backend (серверная часть). Архитектура организована с учётом масштабируемости, безопасности и удобства разработки. Все компоненты разворачиваются в изолированных Docker-контейнерах.

Общая схема архитектуры



Frontend (React + TypeScript)

Клиентская часть реализована с использованием библиотеки React и TypeScript. Она отвечает за:

- отрисовку пользовательского интерфейса;
- маршрутизацию (react-router-dom);
- взаимодействие с REST API;
- хранение токена аутентификации (localStorage);
- валидацию форм, отображение уведомлений (antd, message, Form и др.).

Основные страницы:

- `/` — список популярных фильмов;
- `/movies/:id` — подробности фильма + обзоры и оценка;
- `/watchlist` — список избранных фильмов;
- `/ratings` — просмотр своих оценённых фильмов;
- `/login` и `/register` — аутентификация;
- `/profile/edit` — изменение email и пароля.

Ключевые технологии:

- React (v18+)
- TypeScript
- Ant Design — UI-библиотека;
- Axios — HTTP-запросы;
- JWT — хранение токена;
- React Router — маршрутизация;
- Docker — для контейнеризации сборки.

Backend (Go + chi)

Серверная часть написана на языке **Go** и реализует REST API. Используется минималистичный маршрутизатор **chi**.

Основные модули

- **auth** — регистрация, вход, генерация JWT-токенов;
- **movies** — загрузка популярных фильмов, получение информации по ID;
- **watchlist** — список избранных фильмов пользователя;
- **ratings** — сохранение и просмотр пользовательских оценок;
- **user** — получение и изменение профиля (email, пароль);
- **reviews** — загрузка видеорецензий с YouTube API по названию фильма.

Бизнес-логика разделена на уровни

- **handlers/** — HTTP-обработчики, принимают запросы, валидируют входные данные и формируют HTTP-ответы;
- **service/** — сервисная логика;
- **repo/** — взаимодействие с PostgreSQL (используется **database/sql + sqlx**);
- **models/** — описания структур данных (User, Movie, Rating, Watchlist и т.п.);
- **middleware/** — JWT-аутентификация и обработка ошибок.

База данных

- **PostgreSQL**
- Хранятся таблицы: **users**, **watchlist**, **ratings**, **movies**
- Связь между пользователями и фильмами реализована через **user_id**, **movie_id**

Документация API (Swagger / OpenAPI)

Для удобства разработки и тестирования был подключён генератор спецификации OpenAPI (Swagger).

- **UI-документация:** развернута Swagger UI по пути **/docs/index.html**, где можно интерактивно посылать запросы ко всем эндпоинтам.
- Описаны все модели запросов/ответов, параметры в пути и запросе, схемы ошибок.

Взаимодействие с внешними API

- **YouTube API** — через отдельный клиент `ytClient` осуществляется поиск видеообзоров по названию фильма и извлечение embed-ссылок.
- **Kinopoisk API (неофициальный)** — клиент `kpClient` запрашивает популярные фильмы, постеры, рейтинги. Первоначальная загрузка данных в БД выполняется через batch-скрипты.

Безопасность

- Используется JWT-аутентификация (`Bearer token`) для всех приватных маршрутов. Срок действия - 1 час.
- Хэширование паролей с помощью `bcrypt`.
- Защищённые endpoint'ы с middleware-проверкой токена (`middleware.JWTAuth`).
- Проверка входных данных (валидация на frontend и backend).

Docker-инфраструктура

Все компоненты запускаются через Docker Compose:

- `frontend` — React frontend;
- `api` — сервер Go;
- `postgres` — PostgreSQL;

Контейнер api: собирается по Dockerfile , внутри располагается Go-приложение;

Контейнер frontend: React-приложение собрано в статические файлы и отдается через Nginx;

Контейнер postgres: запускает PostgreSQL с автоматической инициализацией схемы из SQL-скриптов.

Определён `Dockerfile` и `.env` для конфигурации среды.

Ход работы

Разработка приложения велась по этапной схеме, соответствующей типичному жизненному циклу веб-приложений. Каждый этап включал в себя конкретные задачи, технологические решения и промежуточные результаты.

1. Постановка задачи

На первом этапе была изучена и проанализирована предметная область — система подбора и оценки фильмов. Сформулированы требования:

- Регистрация и вход пользователя
- Просмотр списка популярных фильмов
- Добавление фильмов в личный список "К просмотру"
- Возможность оценки фильмов
- Интеграция с внешними источниками (YouTube API)
- Редактирование профиля (email/пароль)
- Поиск фильмов
- Контейнеризация

2. Проектирование архитектуры

Была выбрана архитектура "клиент-сервер", где:

- **Frontend** — SPA-приложение на React с использованием Ant Design
- **Backend** — HTTP API на Go с маршрутизацией (chi), бизнес-логикой и слоями хранилища
- **БД** — PostgreSQL

Разработана структура проекта:

- Чёткое разделение слоёв: `handlers` → `service` → `repository`
- Реализация авторизации на основе JWT
- Использование middleware для аутентификации

3. Разработка backend-части

Реализованы следующие REST API-эндпоинты:

Метод	Путь	Описание
POST	/api/register	Регистрация нового пользователя
POST	/api/login	Аутентификация, получение JWT
GET	/api/users/me	Получение профиля текущего пользователя
PATCH	/api/users/me	Обновление email или пароля

GET	/api/users/{userID}/watchlist	Просмотр избранного списка
POST	/api/users/{userID}/watchlist	Добавление фильма в watchlist
DELETE	/api/users/{userID}/watchlist/{movie_id}	Удаление фильма из watchlist
GET	/api/users/{userID}/ratings	Просмотр пользовательских оценок
POST	/api/users/{userID}/ratings	Добавление/обновление оценки
DELETE	/api/users/{userID}/ratings/{movie_id}	Удаление оценки фильма
GET	/api/movies	Список популярных фильмов
GET	/api/movies/{id}	Детальная информация о фильме
GET	/api/movies/{id}/reviews	Видеообзоры с YouTube по названию фильма
GET	/api/docs	Получение документации Swagger

Дополнительно:

- Все пароли хешируются с помощью **bcrypt**
- Обработка ошибок оформлена с понятными HTTP-кодами и сообщениями

4. Разработка frontend-части

Использован **React + TypeScript** с маршрутизацией на **react-router-dom**. Основные компоненты:

- **LoginPage, RegisterPage**
- **MoviesPage** — просмотр популярных фильмов
- **MovieDetailsPage** — карточка фильма с YouTube-обзорами, оценкой и кнопкой "в список"
- **WatchlistPage** — список к просмотру
- **RatingsPage** — просмотр ранее поставленных оценок
- **EditProfilePage** — смена email и/или пароля

Применялись современные практики:

- Разделение на компоненты
- Асинхронные запросы через **Axios**
- Хранение токена в **localStorage**
- Состояние — через **useState, useEffect**
- Дебаунс для поиска
- Валидация форм

5. Интеграция с внешними сервисами

Была реализована интеграция с **YouTube API**, которая позволяет:

- Автоматически искать обзоры по названию фильма
- Отображать видео-обзоры прямо на странице фильма

Это расширило функциональность платформы и сделало опыт пользователя более насыщенным.

Также была реализована интеграция с **Kinopoisk API**, которая позволяет:

- Автоматически искать фильмы, их рейтинг и остальную важную информацию
- Заполнять предварительно базу фильмами

6. Тестирование и отладка

Unit - тесты

Было реализовано тестирование нижних слоев проекта с помощью встроенного инструментария `go`, библиотеки `testing`. Реализованы автотесты для следующих составляющих проекта:

- Авторизация
- Взаимодействие с внешними API
- Взаимодействие с БД

В тестировании применялись `mock`-объекты для тестирования отдельных блоков независимо друг от друга. Это позволило убедиться в том, что отдельные составляющие ведут себя в соответствии с ожидаемым поведением и работают корректно.

Интеграционные тесты

Интеграционные тесты прогоняются в поднятом стеке (PostgreSQL + API).

Для локального запуска реализован скрипт `run.sh`, который:

- Останавливает и удаляет все контейнеры и тома (`docker compose down -v`).
- Поднимает контейнер с БД и ждёт его статуса `healthy`.
- Сборка образов `builder`, `api` и `frontend`.
- Подъём `api` и ожидание эндпоинта `/health` или `/movies?page=1&size=1`.
- Запуск интеграционных тестов внутри контейнера `builder`:

7. Контейнеризация и развертывание

Dockerfile

1. Stage 1: builder

- Базовый образ: `golang:1.23-alpine`.
- Кэшируются модули (`go mod download`), потом собираются два бинаря:

- `server` (API)
- `import_all_movies` (CLI-утилита для предзагрузки данных - заполнения базы фильмов)

2. Stage 2: runtime

- Базовый образ: `alpine:3.18 + ca-certificates, postgresql-client, curl`.
- Копируются собранные бинарники, файл `.env`, папка `docs`.
- `ENTRYPOINT` указывает на `/app/server`.

`docker-compose.yml`

- **postgres**
 - Образ `postgres:13`, монтируется том с данными и скриптом `init.sql` для инициализации схемы.
 - `healthcheck` через `pg_isready`.
- **api**
 - Собирается из Dockerfile (runtime).
 - Ждёт PostgreSQL, затем запускает `import_all_movies` и стартует `server`.
 - `healthcheck` на эндпоинт `/health`.
- **builder**
 - Собирается из Dockerfile (builder).
 - Ждёт только PostgreSQL.
 - Запускает unit-тесты и затем интеграционные (с помощью переменной `API_URL="http://api:8080"`).
- **frontend**
 - Собирается отдельным Dockerfile из директории `frontend`.
 - Зависит от готовности `api`.

Запуск одним скриптом

Скрипт `run.sh` автоматизирует локальную разработку и тестирование:

Сброс контейнеров/томов → 2. Подъём БД → 3. Сборка образов → 4. Запуск тестов → 5. Старт API и фронтенда.

CI/CD (GitHub Actions)

Конфигурация разбита на два job'а — unit и integration tests. Интеграционные тесты выполняются через Docker Compose по тому же `docker-compose.yml`, что и локально, что обеспечивает идентичность окружений.

8. Финальная доработка

В последней стадии были добавлены:

- Возможность редактирования профиля
- Проверка силы пароля
- Унифицированный дизайн с использованием Ant Design

- Улучшения UX (спиннеры, сообщения, очистка форм)

Планы на будущее / Возможности расширения

В рамках проекта был реализован базовый функционал системы подбора и оценки фильмов. Однако для повышения удобства пользователей, масштабируемости и коммерческого потенциала система может быть значительно расширена. Ниже приведены предложения по развитию проекта:

1. Рекомендательная система

Цель: автоматическое предложение фильмов на основе пользовательских предпочтений.

Что можно сделать:

- Алгоритм коллаборативной фильтрации (на основе оценок других пользователей)
- Фильтрация по жанрам, странам, режиссёрам
- История просмотров и обучения модели по ней

2. Фильтрация и сортировка

Цель: улучшить пользовательский опыт при работе со списками фильмов.

Что можно реализовать:

- Фильтрация по жанрам, году, рейтингу
- Сортировка по дате добавления, популярности, алфавиту
- Быстрый переключатель вида (плитка / таблица)

3. Комментарии и обсуждения

Цель: дать пользователям возможность обмениваться мнениями.

Как реализовать:

- Комментарии к фильмам
- Лайки / дизлайки на комментарии
- Модерация и жалобы

4. Избранное и история просмотров

Цель: предоставить больше персонализации.

Что можно добавить:

- Список "Избранное"
- История всех просмотренных фильмов
- Метка "Просмотрено"

5. Социальные функции

Цель: сделать приложение более интерактивным.

Что можно реализовать:

- Добавление друзей
- Подписки на профили

- Возможность видеть, что смотрят и оценивают другие

6. Адаптивная и мобильная версия

Цель: доступность на всех устройствах.

Что нужно сделать:

- Полноценная мобильная адаптация
- Поддержка PWA (прогрессивного веб-приложения)
- React Native-приложение с тем же API

7. Административная панель

Цель: управление системой и её пользователями.

Что предусмотреть:

- Удаление фильмов и комментариев
- Статистика (посещаемость, самые просматриваемые фильмы)
- Управление пользователями и правами

8. Интеграция сторонних API

Цель: расширить источники данных и повысить качество.

Варианты:

- IMDb или TMDb API для альтернативных метаданных
- Telegram-бот для напоминаний о фильмах
- Календарь новинок (с расписанием премьер)

9. Мультиязычность

Цель: сделать сервис доступным для пользователей других стран.

Как реализовать:

- Поддержка i18n (напр., библиотека `react-i18next`)
- Перевод интерфейса и описаний

10. Монетизация

Цель: подготовка продукта к коммерческому использованию.

Идеи:

- Премиум-аккаунты (доступ к расширенным функциям)
- Реклама (с учётом рейтингов)
- Партнёрство с онлайн-кинотеатрами

Вывод

В результате выполнения курсового проекта было разработано полнофункциональное веб-приложение «**Movie Picker**» — сервис подбора, оценки и сохранения фильмов. Работа над проектом включала проектирование, реализацию и тестирование как клиентской, так и серверной части, что позволило получить целостную и масштабируемую архитектуру.

Соответствие поставленной задаче

Проект полностью соответствует исходной постановке задачи:

- Пользователи могут регистрироваться и авторизовываться
- Доступен просмотр популярных фильмов и поиск по названию
- Можно добавлять фильмы в список «К просмотру»
- Реализовано выставление оценок и просмотр собственных рейтингов
- Присутствует интеграция с YouTube API для получения обзоров
- Доступна возможность редактировать личный профиль (email, пароль)

Таким образом, базовая функциональность реализована в полном объёме, с удобным пользовательским интерфейсом и защищённой серверной логикой.

Технические итоги

- **Фронтенд** реализован на **React** с использованием **TypeScript**, библиотеки **Ant Design** и клиентского роутинга (**React Router**).
- **Бэкенд** написан на **Go**, с использованием **Chi** в качестве маршрутизатора, **PostgreSQL** как СУБД, и **bcrypt** для безопасного хранения паролей.
- Реализована авторизация по **JWT**, взаимодействие с API, работа с YouTube-обзорами и обработка ошибок.
- Приложение успешно разделено на публичную и защищённую части с проверкой токена.

Полученные навыки и опыт

В ходе выполнения проекта были получены ценные практические навыки:

- Проектирование RESTful API и клиент-серверной архитектуры
- Работа с внешними API (в частности, YouTube Data API)
- Управление состоянием в React-приложении
- Создание защищённой системы аутентификации и авторизации
- Уверенное использование современных инструментов frontend и backend-разработки

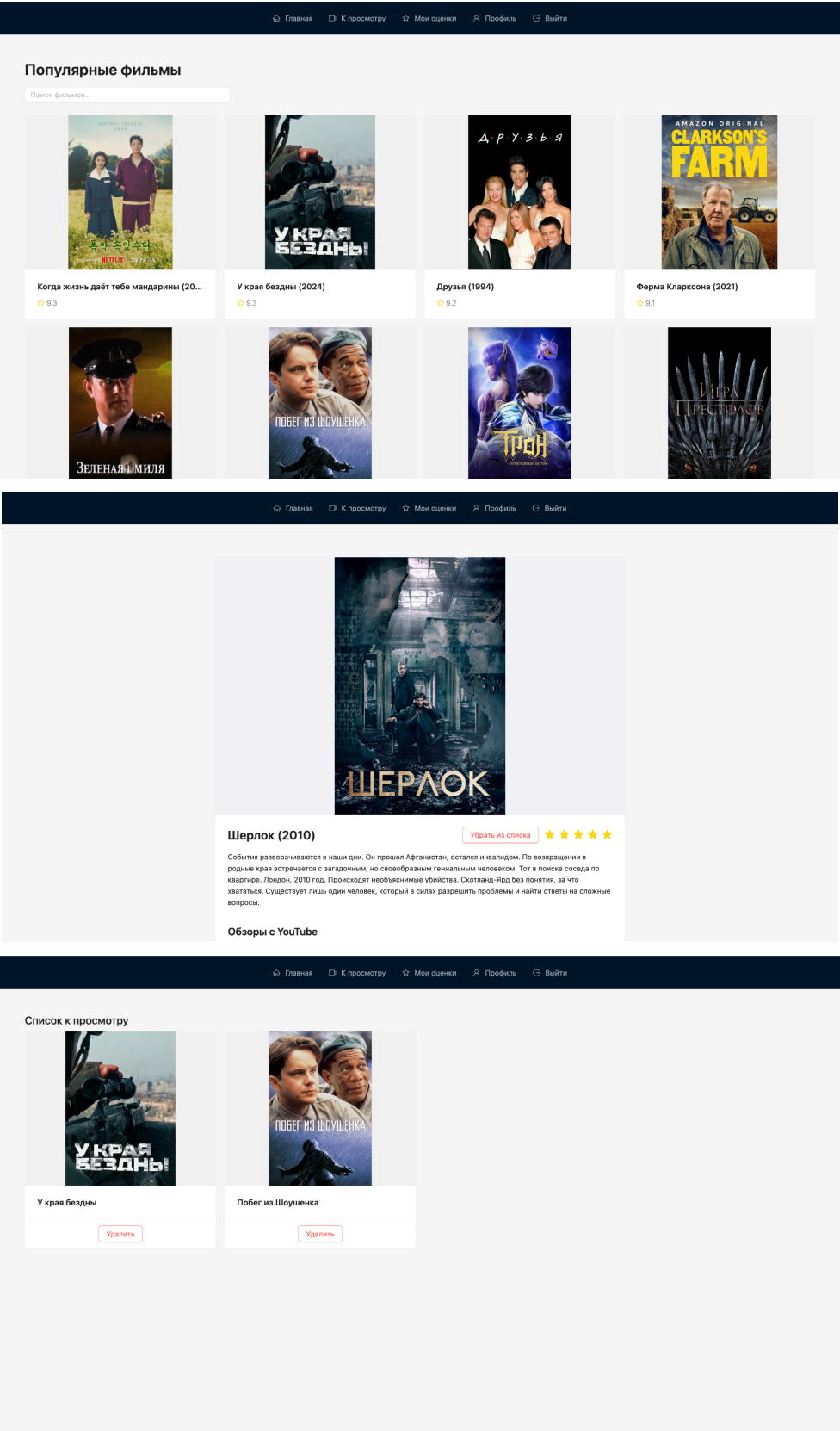
Заключение

Приложение можно считать успешно реализованным. Оно стабильно функционирует, предоставляет пользователю удобный интерфейс для работы с фильмами и демонстрирует правильную архитектурную реализацию как со стороны клиента, так и сервера.

Проект может быть использован как основа для более крупного и коммерчески ориентированного продукта.

Приложения

Скриншоты приложения



ГлавнаяК просмотруМои оценкиПрофильВыйти

Мои рейтинги

У края бездны (2024)
★★★★★ (5)

Побег из Шоушенка (1994)
★★★★★ (5)

Шерлок (2010)
★★★★★ (5)

Редактировать профиль

Email

svetlana.vdovina2004@gmail.com

Новый пароль

Оставьте пустым, если не меняется

* Текущий пароль

Обязательно для изменения email или пароля

Сохранить изменения

Если вы хотите сменить email или пароль — введите текущий пароль.

Вход

* Email

* Пароль

Войти

Нет аккаунта? [Зарегистрируйтесь](#)

Регистрация

* Email

* Пароль

Зарегистрироваться

Уже есть аккаунт? [Войти](#)